

Evaluating a GDDR6-AiM-TPU Hybrid Architecture for Deep Learning

Erik Xie
ejxie@mit.edu

Andy Yu
andy455@mit.edu

I. INTRODUCTION

Modern LLM inference exhibits two different computational phases: prefill, where large batches of tokens are processed in parallel through compute-intensive matrix-matrix multiplications, and decode, where tokens are generated one at a time through memory-bound matrix-vector operations. Due to architectural specialization, TPUs-style accelerators work well for prefill workloads, whereas in-memory accelerators (AiM) are optimized for the decode stage. This paper looks at whether a hybrid architecture combining a TPU with a GDDR6-AiM accelerator behind a shared DRAM will allow the system to process LLM workloads more efficiently. We expect memory-bound einsums in LLM workloads (i.e. softmax, matvec multiplications) to have lower EDP (latency-energy product) on the GDDR6-AiM accelerator than on TPU, and vice versa, and that the overall EDP is reduced compared to a TPU-only architecture. We will validate these tradeoffs via a few experiments in this paper.

II. RELATED WORKS

Large language models (LLMs) based on the transformer architecture have revolutionized the field of artificial intelligence in recent years [25]. Consequentially, a large variety of hardware accelerators have been developed to meet the growing demand for LLM training and inference [17]. Many such demands involve a shift from compute-centric architectures to memory-centric architectures because of the massive scaling in compute technologies that isn't followed by proportionate scaling in memory bandwidth [20]. This gap is known as the "Memory Wall" [24] and remains one of the critical bottlenecks in many conventional computation platforms like GPUs, TPUs, and other custom accelerators for Deep Neural Networks (DNNs).

The memory wall is particularly problematic for modern LLM inference given its prefill and decode differences [10]. Many solutions have been proposed. Generalist GPUs are highly parallel architectures that can process a wide range of workloads, not limited to LLMs. On the software level, GPU frameworks like vLLMs [9] introduce PagedAttention to divide key-value attention into fixed-size blocks to optimize memory access. Similarly, TurboTransformers [3] employs segment-based memory management to handle variable-length inputs, while ALISA [30] leverages sparse attention to balance memory access. On the other hand, traditional LLM accelerators like Google's TPU rely on hardware like systolic arrays

that are highly optimized for the compute-intensive matrix-matrix multiplications of the prefill phase [15]. However, during the decode phase, where tokens are generated sequentially via matrix-vector operations, these compute-centric architectures leave significant memory bandwidth underutilized [29]. This mismatch results in extra latency and show the energy cost of moving data between the computation units and off-chip memory [13].

In this context, Accelerator-in-Memory (AiM) has been proposed to overcome bandwidth limitations, data access latencies, and data movement energy costs [20]. The key idea of AiM is to integrate computation units directly inside the memory to exploit internal data parallelism [20]. Among different memory technologies, DRAM has become one of the most heavily studied due to its high capacity, endurance, and technological maturity [13]. Consequently, various DRAM models have been adapted for AiM architectures: DDRx for general-purpose applications [13], LPDDRx for low-power applications [4], GDDRx for graphics applications (e.g., GDDR5 and GDDR6) [8], and HBMx for high-bandwidth applications [17]. These architectures can be further divided into device-based [17] or 2.5/3D stacked configurations [16], with stacked architectures leveraging separate manufacturing processes to incorporate more efficient compute units [13].

Because LLM inference has mixed computational demands, some proposed in-memory solutions have adopted specialized or hybrid internal structures to maximize efficiency. For example, the X-Former architecture [18] combines Non-Volatile Memory (NVM) processing elements with CMOS processing tiles. It delegates memory-heavy computations to the NVM, while utilizing the CMOS tiles to handle more demanding self-attention layers. Similarly, H3DAtten [11] integrates both analog and digital compute-in-memory (CIM) to balance energy consumption and latency. Other architectures, such as TranCIM [23], rely on a fully digital CIM approach to optimize computations everywhere.

Recent advancements in pure AiM architectures, such as the GDDR6-based AiM [8], highlight the advantages of placing computation units in the peripheral region of the memory bank. This placement allows the device to operate concurrently as both main memory and an energy-efficient accelerator without changing the memory array design [7]. However, while GDDR6-AiM architectures offer the massive internal DRAM bandwidth necessary for memory-bound operations, they still lack the throughput required for compute-heavy

tasks [6]. Because LLM workloads consist of einsums with wildly varying computational intensities, we build upon the premise that hybrid architectures are more suited for end-to-end inference. This paper investigates a novel hybrid architecture designed to bridge this gap.

III. APPROACH

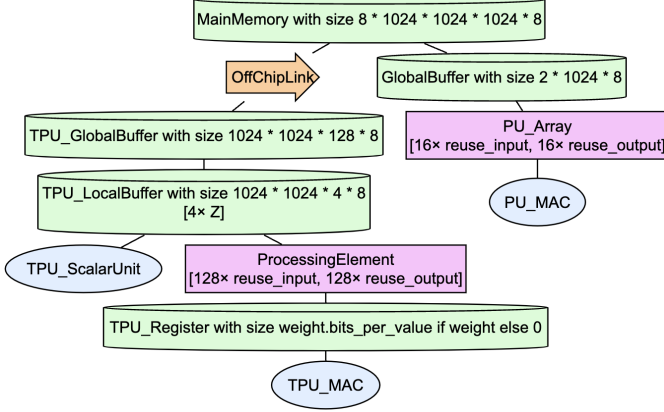


Fig. 1. Hybrid Architecture with TPU and AiM

Our approach combines these two models (TPU, GDDR6-AiM) into a hybrid architecture that leverages both compute-centric acceleration (TPU) and memory-centric acceleration (AiM). The hybrid architecture is modeled with an area of 211.5950 mm², where the TPU only architecture is 208.5536 mm², so including an AiM component only incurs about 1.46% area over TPU-only. The goal is to exploit their advantages given this minimal area cost, mapping compute-bound operations to the TPU while offloading memory-bound operations to GDDR6-AiM. This differs from previous works that typically evaluates these architectures in isolation [15], or explores other AiMs. We will compare three configurations: (1) TPU-only, (2) AiM-only and (3) the hybrid TPU+AiM architecture. All three share the same main memory, so that the computational differences come only from the on-chip compute and memory hierarchy. The two non-hybrid architectures will therefore serve as the baselines. The hybrid architecture is carefully modeled (see Appendix) and visualized in figure 1. We use a GPT-3 workload modeled with einsums for our experiments. The rank sizes correspond to a 6.7B model size with KV cache. We conduct three experiments to assess the performance and tradeoffs of this system.

IV. EXPERIMENT SETUP

To evaluate the proposed architecture, we use an automated mapper [2] to find the optimal mapping for energy and latency. To ensure fair comparison, we report the Energy-Latency-Area-product (EDAP) when applicable, which accounts for the area of the accelerator. We design the following experiments to evaluate this system:

Experiment 1: Energy, latency, and throughput analysis. We evaluate how workload and sequence length impacts performance and hardware mapping by sweeping the N_TOKENS

parameter across 1024, 4096 (4×1024), and 8192 (8×1024) for the decode and prefill workloads. For each N_TOKENS, we run the workload on all three architectures and record the per-einsum energy, latency, and throughput. For the hybrid model, we specifically track the mapper’s hardware selection (TPU vs. AiM) for later analysis.

Experiment 2: Impact of Fusion. To understand how memory traffic affects architectural preferences, we map the decode and prefill workloads onto all three architectures under two configurations: fusion enabled and fusion disabled. This isolates the effect of intermediate tensor write-backs on the mapper’s decision to route specific Einsums to either the TPU or AiM in the hybrid architecture. We again track the mapper’s hardware selection (TPU vs. AiM) for later analysis.

Experiment 3: Architecture Exploration. To identify the optimal hardware balance for the hybrid architecture, we sweep two key design parameters: Processing Unit (PU) count (16, 64, 128, and 256) and external memory bandwidth (8 to 126 GB/s). We measure energy and latency for both prefill and decode workloads to analyze the bottleneck transitions between on-chip compute and off-chip memory.

V. RESULTS AND DISCUSSION

A. Per Einsum Analysis over N_TOKENS

In this experiment, it is demonstrated that the AiM has superior energy, latency, and throughput with non-computationally intensive workloads, while the TPU excels with computationally intensive workloads with more reuse opportunities. Figures 2 and 3 show the energy, latency, and throughputs per Einsum for different N_TOKENS values. The throughputs are plotted with the AiM and TPU roofline curves.

From Fig. 2, the AiM architecture’s energy and latency per Einsum is consistently lower than the TPU’s in the decode stage. Due to the low computational intensity or low reuse opportunities in the decode stage, the workload is memory bound. Therefore, the AiM’s higher memory bandwidth is always better, despite the TPU’s superior computational throughput, so the hybrid architecture only uses the AiM.

This is reinforced in Fig. 2’s roofline plots, where the Einsums’ optimal throughputs lie on the AiM’s higher roofline curve in the lower computational intensity region.

Also shown in Fig. 2, only the energy and latency of Einsums QK, QK_softmax, and AV scale with N_TOKENS for Einsums QK, QK_softmax, and AV. This is expected, as these are the only Einsums whose dimensions are determined by N_TOKENS in the decode stage.

In Fig. 3, it is shown that the AiM architecture now has consistently higher energy and latency per Einsum than the TPU in the prefill stage. The prefill stage is more computationally intensive than the decode stage, with weights reused across $N_NEW_TOKENS = N_TOKENS$. Therefore, the AiM architecture becomes “over utilized” and inefficient when compared to the TPU’s reuse-optimized architecture. Nonetheless, I and QK_softmax are still computationally light enough such that the AiM remains optimal over TPU. Therefore, every Einsum

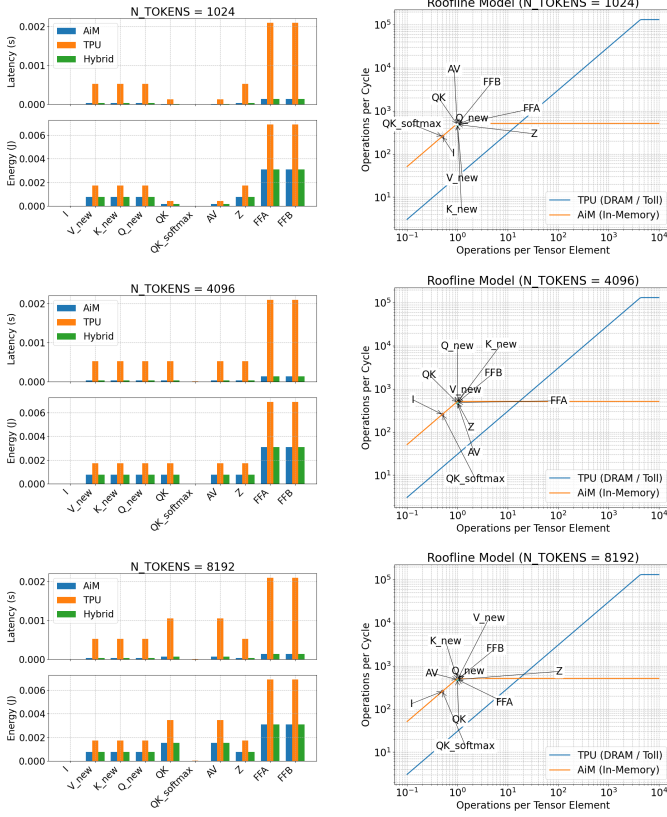


Fig. 2. Per Einsum energy and latency for each N_TOKENS value.

except I and QK_softmax are mapped to the TPU in the hybrid architecture.

This is further demonstrated in the roofline plots in Fig. 3. Every Einsum except I and QK_softmax have higher computational intensities and have throughputs that lie on the TPU’s roofline curve, which is optimal for higher computational intensities. Additionally, it is shown that the computational intensities of Z, K_new, Q_new, V_new, FFA, and FFB increases with N_TOKENS. This is expected, because as N_TOKENS = N_NEW_TOKENS increases, identical weights are reused to generate additional tokens, which increases the computational intensity of the involved Einsums listed before.

Like in the decode stage, the energy and latency values for QK, QK_softmax, and AV scale with N_TOKENS. However, since N_NEW_TOKENS = N_TOKENS, the metrics for every other Einsum now scales with N_TOKENS (note the different y-axes), as these Einsums have a dimension determined by N_NEW_TOKENS in the workload.

These results visualize the superiority of the AiM and TPU architectures in low and high computational intensity workloads respectively. Such differences justify the mapping of each Einsums to particular architectures in the decode and prefill stages without fusion, suggesting the hybrid architecture’s potential benefits over each architecture individually.

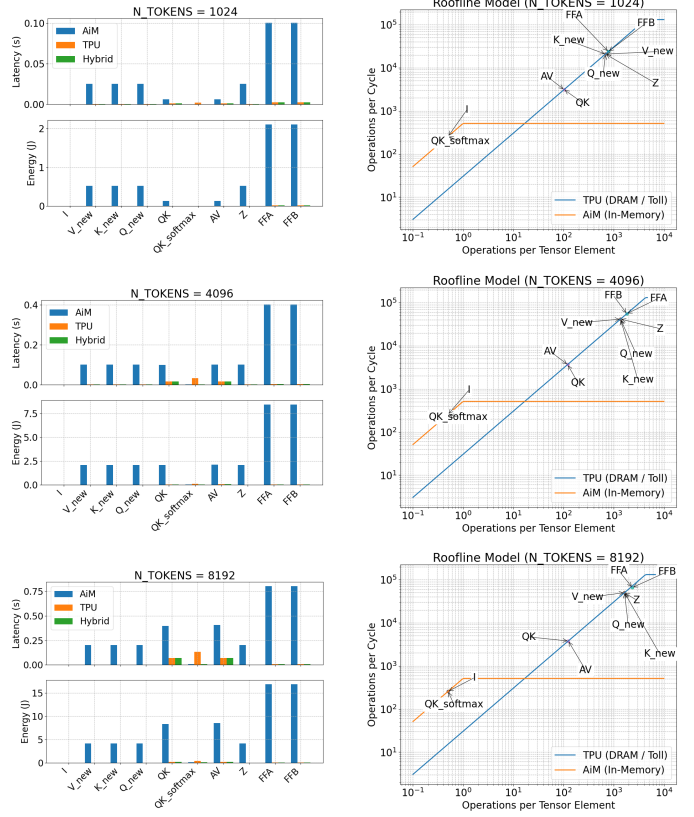


Fig. 3. Per Einsum energy and latency for each N_TOKENS value.

B. No Fusion vs. Fusion

In the hybrid architecture, it is shown that fusion provides little to no benefit in workloads with small intermediate tensors that provide little reuse opportunities, like the decode stage. For more intensive workloads, like the prefill stage, fusion is shown to improve energy and latency metrics. Furthermore, it is shown that AiM does not benefit from fusion, while the TPU benefits from fusion.

The hybrid architecture mapping of the decode workload is the same without or without fusion. The energy and latency metrics of each mapping are shown in Table I. In the mapping, all computations are done with the AiM. The identical values between the Hybrid and AiM architectures in Table I confirm this. Furthermore, the identical metrics for the AiM with and without fusion suggests that the AiM does not notably benefit from fusion. The table also shows the inefficiency of the TPU in the decode stage, with its energy and latency being higher than the AiM’s, with fusion making a slight improvement.

These results are likely due to the low data reuse opportunities and small intermediate tensor sizes in the decode stage. As shown in Experiment 1, less reuse opportunities benefits the AiM due to its high memory bandwidth. Furthermore, small intermediate tensor sizes provide less opportunities for reuse, which diminishes potential benefits of fusion. This is essentially non-consequential for the AiM, as its 2KB global buffer is not best suited for reuse regardless. However, the

TABLE I
DECODE FUSION PERFORMANCE METRICS

arch	fusion	total energy (J)	total latency (s)	edp (J*s)
Hybrid	fused	0.012397	0.000559	6.930e-6
Hybrid	unfused	0.012397	0.000559	6.930e-6
AiM	fused	0.012397	0.000559	6.930e-6
AiM	unfused	0.012397	0.000559	6.930e-6
TPU	fused	0.027682	0.008390	0.000232
TPU	unfused	0.027774	0.008424	0.000234

TPU’s reuse capabilities would be underutilized and inefficient if used in the decode stage. Therefore, fusion only provides a small energy and latency improvement for the TPU. These reasons again show why, in the decode stage, why the AiM shows no difference in metrics with or without fusion and why the AiM remains preferable to the TPU. Therefore, even though the TPU is optimized to take advantage of fusion, a workload with insufficient computational intensity and intermediate tensor sizes leads the AiM to remain optimal with or without fusion on the hybrid architecture.

The hybrid mapping of the prefill workload with and without fusion are shown in Fig. 4. The energy and latency metrics are shown in Table II. Note that the mappings are now visually different with and without fusion. The better values of the Hybrid architecture compared to both the AiM and TPU in Table II indicates that the optimal mapping without fusion uses both architectures. This is expected and corroborates Experiment 1, where both architectures were assigned different Einsums for the prefill workload without fusion. The identical values between the TPU and Hybrid with fusion in Table II indicate that all computations are mapped to the TPU when fusion is enabled.

The inefficiency of the AiM in the prefill stage is plainly shown by its considerably higher energy and latency metrics compared to the TPU in Table II. Additionally, the AiM’s metrics show no change between fusion and no fusion, while the TPU again demonstrates improvements when fusion is enabled.



Fig. 4. Prefill Mapping (Left: No Fusion; Right: Fusion)

These results show that for workloads with high data reuse opportunities and large intermediate tensors, like the prefill stage, the TPU vastly outperforms the AiM. This is due to the TPU’s memory hierarchy and higher computational capacity that allows it to efficiently reuse values and intermediate tensors. As stated before, the AiM lacks the on-chip memory to perform fusion efficiently, limiting its use case to Einsums with

TABLE II
PREFILL FUSION PERFORMANCE METRICS

arch	fusion	total energy (J)	total latency (s)	edp (J*s)
Hybrid	fused	0.562860	0.037000	0.020826
Hybrid	unfused	1.033843	0.173396	0.179264
AiM	fused	67.66759	3.229745	218.5491
AiM	unfused	67.66759	3.229745	218.5491
TPU	fused	0.562860	0.037000	0.020826
TPU	unfused	1.312323	0.300142	0.393883

low computational intensity. Therefore, the prefill workload is split when fusion is not enabled, because the AiM is still better for Einsums with low computational intensity, which cannot be passed as intermediate tensors with fusion disabled. However, with fusion enabled, the TPU’s architecture is much better utilized such that it is preferable for all computations and potential intermediate tensors to be calculated and stored on the TPU respectively. This circumvents the repeated cost of accessing the off-chip DRAM, and leads the mapper to allocate all computations to the TPU with fusion enabled. Therefore, as computational intensity and intermediate tensor sizes increase, the TPU becomes more preferred over the AiM, especially with fusion enabled. Lastly, there were some reuse opportunities missed by the hybrid architecture due to the fact that intermediates between einsums mapped to different subunits must pass through DRAM, but this is shadowed by the improved latency of the hybrid mapping.

C. AiM Architecture Exploration

In this experiment, it is shown that the hybrid architecture can be improved with more AiM PUs, trading off area, and more external bandwidth, trading off cost, and both hit diminishing returns up to some amount. To validate this, our architecture exploration is grounded on the different bottlenecks of the TPU and AiM subunits of the system. Based on our model of the architecture, we define L_{ext} and L_{int} to be the latency of the DRAM’s internal and external traffic. The maximum throughput of each unit is (TPU, AiM):

$$16,384 \text{ parallel units} \times 1.05 \text{ GHz} = 17.2 \text{ TMACs/s}$$

$$256 \text{ units} \times 16 \text{ GHz} = 4.1 \text{ TMACs/s}$$

To sustain these computational ceilings without being memory-bottlenecked, the bandwidth must satisfy the arithmetic intensity of the mapped einsums. The external link (L_{ext}) represents a severe physical bottleneck for the TPU, bounded at 32 GB/s at the default configuration. And the AiM only has 256 PUs, which is only $\sim 1\%$ of TPU’s throughput. To validate these bottlenecks, we mathematically model one representative einsum, a matrix-vector multiplication, which is memory intensive:

$$Q_{new}[b, m, h, e] = I[b, m, d] * WQ[h, e, d]$$

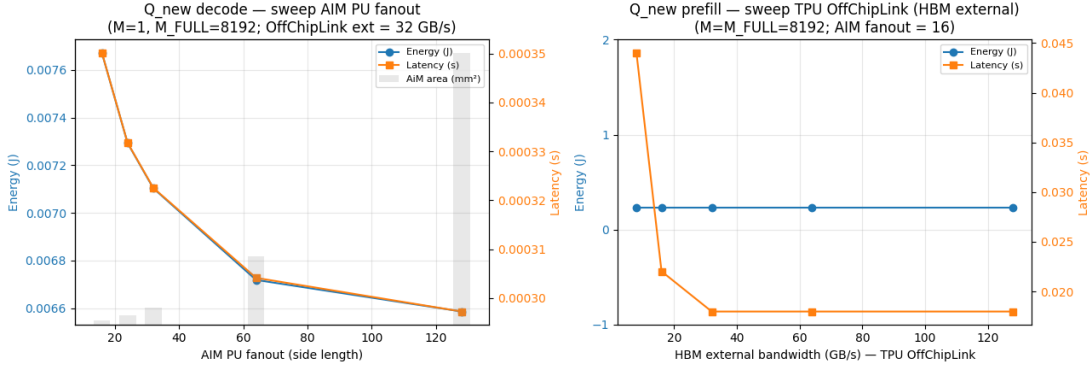


Fig. 5. PU fanout and DRAM Bandwidth sweep

It has a data movement of around 16.78×10^6 Bytes and a compute requirement of 16,777,216 during decode. If mapped to the AiM during prefill, the execution time is

$$T_{AiM,Q} = \max \left(\frac{16.77 \times 10^6}{4.1 \times 10^{12}}, 8192 \cdot L_{int} \right)$$

$$T_{AiM,Q} = \max (4.09 \mu s, 8192 \cdot L_{int})$$

The AiM becomes compute-bound if $L_{int} > \frac{4.09 \mu s}{8192 B}$, meaning the internal bandwidth would only need to exceed 2.0 GB/s. Given the massive internal bandwidth of the PUs, the AiM is all compute-bound during decode. Increasing PU count could be a profitable choice here.

During prefill, the sequence length enables massive data reuse, and the compute requirement grows to 137.4 Giga-MACs, while the total external data transferred becomes roughly 83.88 MB. The TPU execution time becomes:

$$T_{TPU,Q_{prefill}} = \max \left(\frac{137.4 \times 10^9}{17.2 \times 10^{12}}, 83.88 \times 10^6 \cdot L_{ext} \right)$$

$$T_{TPU,Q_{prefill}} = \max (7.99 \text{ ms}, 83.88 \times 10^6 \cdot L_{ext})$$

This shows the memory boundary at a threshold: $L_{ext} > \frac{7.99 \text{ ms}}{83.88 \text{ MB}}$, or approximately 10.5 GB/s. If we sweep the TPU's external bandwidth around this range during the prefill phase, we can observe the TPU's memory bottleneck shift to a compute one during.

Figure 5 Shows the simulated results where the PU fanout and external bandwidth are swept over a few possibilities. They validate the bottleneck thresholds we derived in the previous steps. In the case of AiM during the decode phase, the latency line slopes downward as we add more PUs. Because the weights are stationary inside the AiM, there is no memory penalty. The internal bandwidth is so massive that the system never has to wait for data. The only bottleneck is raw arithmetic throughput which is dictated by PU count. Realistic, the area of the die would grow rapidly as we add more PUs, which is a significant tradeoff. Another one is the diminishing return as PU fanout grows, shown by the decreased slope of the latency line.

Similar to latency, reducing the bottleneck for compute results in a downward energy trend, which can be explained

by increased spatial data reuse. This comes with a significant area cost, which is shown in in figure 5. We can compute the EDAP (energy-delay-area product) for each PU fanout level, which shows an optimum at AiM fanout of 16 by 16 (EDAP = $8.168e-12 \text{ J*s*mm}^2$). Since this is the original GDDR6-AiM paper's configuration, this also validates the author's design choice.

For TPU, we see a similar roofline. With smaller external bandwidth, it is stalled waiting for the external memory and its compute is underutilized. Since the prefill stage has massive tensors, the latency flatlines into the compute-bound stage pretty quickly. Because it crossed the mathematical threshold, the memory can deliver data faster than the TPU can multiply it. The tradeoff is the significant cost of higher bandwidth in DRAM, with HBM memory costing 3-10 times as much as the GDDR6 DRAM used in the modeling (per GB), due to the advanced packaging, bonding techniques, and other manufacturing costs [1]. Since the bandwidth mostly affects latency, the energy stays mostly stationary as shown by the flat line, except some really small fluctuation due to leak power, which is proportional to latency.

VI. LLM USAGE

LLMs were used to assist with the completion of this project. It was useful for creating the figures in the paper, which involves working with font sizes, formatting, and the specifics of graphic libraries. It was unhelpful for researching existing papers on modeling the system due to frequent hallucinations and lack of credible sources.

VII. APPENDIX

A. Modeling Energy of GDDR6 Internal Read/Write Operations

Based on the anatomy of DRAM operation, the energy can be modeled as the following:

$$E = E_{\text{activate}} + E_{\text{read}} + E_{\text{route}}$$

Where the three components are the energy cost of row activation, selecting the specific column, and driving the data to the PU. E_{route} can be computed using the standard CMOS dynamic power equation:

$$E_{\text{route}} = \alpha \cdot c \cdot L \cdot V^2$$

We can measure the distance from each GDDR6 bank to its PU using the die photo provided in the paper. The physical distance is about 15.6% of the height of each bank [8]. Each GDDR6 bank has a specified height of 0.532 mm [20], which gives a distance of 82.99 μm . 0.2 fF/ μm has been shown to be a reasonable estimation of parasitic capacitance across all technology nodes [28]. The authors of GDDR6-AiM specified that the system runs at 1.25V. Since α is the activity factor (chance a bit flips from 0 to 1), we assume $\alpha = 0.25$ since there are 4 possible transitions between 0 and 1 on a wire.

Therefore, we have:

$$\begin{aligned} E_{\text{route}} &= \alpha \cdot c \cdot L \cdot V^2 \\ &= 0.25 \cdot 0.2 \frac{\text{fF}}{\mu\text{m}} \cdot 82.99 \mu\text{m} \cdot (1.25\text{V})^2 = 0.0064 \text{ pJ} \end{aligned}$$

We can compute the row activation energy using the following parameters published in the GDDR6 datasheet [5] and the standard formula [12]. The row activation power isolates the dynamic current used to activate a row by subtracting the background standby currents from the total I_{DD0} measurement. According to the power calculation formulas, the core and pump activation power is calculated as follows:

- Core Activation Power: $P_{\text{core}}(ACT) = \left(I_{DD0} - \left[I_{DD3N} \times \frac{t_{RAS}}{t_{RC}} + I_{DD2N} \times \frac{t_{RP}}{t_{RC}} \right] \right) \times V_{DD}$
- Pump Activation Power: $P_{\text{pump}}(ACT) = \left(I_{PP0} - \left[I_{PP3N} \times \frac{t_{RAS}}{t_{RC}} + I_{PP2N} \times \frac{t_{RP}}{t_{RC}} \right] \right) \times V_{PP}$

To find the row activation energy, we calculate the total power ($P_{\text{core}} + P_{\text{pump}}$) and multiply it by the row cycle time (t_{RC}). The column select (read) power isolates the dynamic current used to fetch data from the open row by subtracting the active standby current (I_{DD3N}) from the total burst read current (I_{DD4R}).

- Column Read Power: $P_{\text{core}}(RD) = (I_{DD4R} - I_{DD3N}) \times V_{DD}$

To obtain the column read energy, we multiply the column read power by the burst duration (t_{burst}). In GDDR6, the burst duration is derived from the data rate and the fixed burst length (BL) of 16. At 16 Gbps, the transmission time per bit is 0.0625 ns, resulting in a t_{burst} of exactly 1.0 ns (16 bits $\times$ 0.0625 ns). For write operations, the read current (I_{DD4R}) is replaced with the burst write current (I_{DD4W}) to find the column write power, which is similarly multiplied by t_{burst} to find the write energy.

Parameter	Description	Value
I_{DD0}	Total Measurement Current (Active-Precharge)	160 mA
I_{DD3N}	Active Standby Current	110 mA
I_{DD2N}	Precharge Standby Current	90 mA
I_{DD4R}	Total Burst Read Current	790 mA
I_{DD4W}	Total Burst Write Current	590 mA
I_{PP0}	Pump Operating Current	30 mA
I_{PP3N}	Pump Active Standby Current	20 mA
I_{PP2N}	Pump Precharge Standby Current	20 mA
t_{RAS}	Active to Precharge Command Period	32 ns
t_{RC}	Row Cycle Time	47 ns
t_{RP}	Precharge Command Period	15 ns
V_{DD}	Core Voltage	1.25 V
V_{PP}	Pump Voltage	1.8 V
$P_{\text{core}}(ACT)$	Core Activation Power	70.5 mW
$P_{\text{pump}}(ACT)$	Pump Activation Power	18.0 mW
E_{RD}	Column Read Energy	850 pJ
$P_{\text{core}}(RD)$	Column Read Power	850.0 mW
E_{WR}	Column Write Energy	600 pJ
$P_{\text{core}}(WR)$	Column Write Power	600.0 mW
E_{ACT}	Row Activation Energy	4159 pJ

These values are per-command. Finally, we normalize our energies to per_bit values. Each column read transfers 256 bits:

$$E_{\text{RD_per_bit}} = 850 \text{ pJ} / 256 \text{ bits} = 3.32 \text{ pJ / bit}$$

Each row activation activates 2048 values [7], assuming locality:

$$E_{\text{ACT_per_bit}} = 4159 \text{ pJ} / 2048 \text{ bits} = 2.03 \text{ pJ / bit}$$

Similarly, for write operations, each column write transfers a burst of 256 bits:

$$E_{\text{WR_per_bit}} = 600 \text{ pJ} / 256 \text{ bits} = 2.34 \text{ pJ / bit}$$

The routing energy calculated earlier ($E_{\text{route}} = 0.0129 \text{ pJ}$) represents the energy to drive a single bit-line transition across the internal bus based on the activity factor α , therefore it is already a per-bit value. Finally, by summing the three

components ($E_{\text{Total}} = E_{\text{ACT_per_bit}} + E_{\text{column_per_bit}} + E_{\text{route}}$), we arrive at the total dynamic energy per bit for read and write operations:

Total Read Energy Per Bit:

$$E_{\text{Total_RD}} = 2.03 \text{ pJ} + 3.32 \text{ pJ} + 0.0064 \text{ pJ} = 5.3564 \text{ pJ / bit}$$

Total Write Energy Per Bit:

$$E_{\text{Total_WR}} = 2.03 \text{ pJ} + 2.34 \text{ pJ} + 0.0064 \text{ pJ} = 4.3764 \text{ pJ / bit}$$

We visualize these energy breakdowns under different assumptions in figure 6.

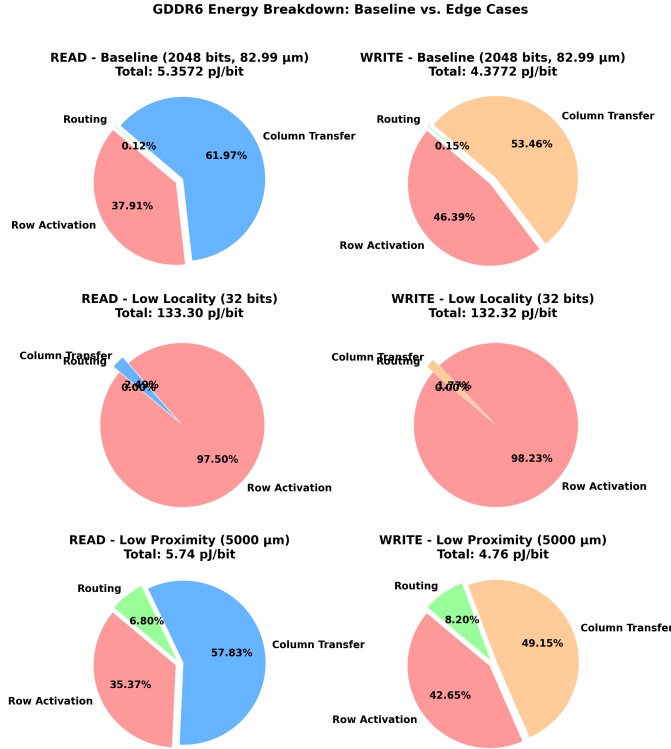


Fig. 6. Breakdown of on-chip read/write energy

B. Modeling Latency of GDDR6 Internal/External Operations

The authors of GDDR6-AiM provide the internal and external bandwidth for their architecture at a size of 8GB DRAM [27]. For on-chip PUs, the internal bandwidth (512GB/s) defines the latency, and for off-chip TPU, the external bandwidth (32GB/s) defines the latency. Integrating compute in-memory drastically reduces the latency, as expected.

C. Modeling Energy of GDDR6 External Operations

According to Chirag et al. [20], GDDR6 off-chip operations have an energy cost of 7.5pJ/bit for TPUv4. We use this number directly in our model.

D. Modeling Energy of AiM MACs

According to Chirag et al. [21], the peripheral transistor performance in a 2y nm DRAM technology is approximately equivalent to a 65 nm logic technology, so we take 1y nm DRAM technology (GDDR6-AiM's manufacturing process) as approximately equivalent to 32 nm logic technology, since the a roughly linear relationship shown in previous literature by Sung et. al. [22].

Since TPUv4 is based on 7 nm logic technology [26] and has known energy/MAC of 0.084 pJ [2], we use empirical CMOS scaling equations [19] to obtain the energy for the 2y nm DRAM manufacturing process. We assume the MAC will operate at the same voltage as the 7nm TPU logic units, which is typically 0.7V [14]. The CMOS scaling equations specify known coefficients for different technology nodes, which we take here:

$$\begin{aligned} \text{EnergyFactor_x} &= 0.8367(0.7)^2 - 0.4341(0.7) + 0.1701 \\ &= 0.276213 \end{aligned}$$

$$\begin{aligned} \text{EnergyFactor_y} &= 0.1776(0.7)^2 - 0.09097(0.7) + 0.02447 \\ &= 0.047815 \end{aligned}$$

$$\text{Scale} = \frac{0.24223375}{0.84433} = 5.7767018718$$

Therefore, we have:

$$\begin{aligned} E_{\text{PU_MAC}} &= 5.7767018718 \cdot E_{\text{TPU_MAC}} \\ &= 5.7767018718 \cdot 0.084 \\ &= 0.48\text{pJ} \end{aligned}$$

E. Modeling Area and Latency of AiM MACs

The area of an AiM PU is given to be 0.19 mm² [8], we divide by 16 to approximate the per MAC area: 0.011875 mm².

The latency is calculated by the paper specified 32GFLOP/s throughput per PU [8]. Each MAC performs 2 FLOPS, therefore, the latency is calculated to be $\frac{1}{1e9 \cdot 16}$.

REFERENCES

- [1] HBM vs. DDR: Key Differences in Memory Technology Explained — IntuitionLabs — intuitionlabs.ai. <https://intuitionlabs.ai/articles/hbm-vs-ddr-memory-comparison>. [Accessed 28-04-2026].
- [2] Tanner Andrulis and Michael Gilbert. Accelforge.
- [3] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbotransformers: an efficient gpu serving system for transformer models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 389–402, 2021.
- [4] Siyuan He, Zhantong Zhu, Yandong He, and Tianyu Jia. Lp-spec: Leveraging lpddr pim for efficient llm mobile speculative inference with architecture-dataflow co-optimization, 2025.
- [5] SK Hynix. 16Gb GDDR6 SGRAM. https://www.utt.com.tw/wp-content/uploads/2025/12/H56G42AXXX014_rev1.1.pdf. [Accessed 17-04-2026].
- [6] Norman P. Jouppi et al. Tpu v4: An optically reconfigurable super-computer for machine learning with hardware support for embeddings. *ISCA*, 2023.
- [7] Daehan Kwon et al. System architecture and software stack for gddr6-aim. [Accessed 17-04-2026].
- [8] Daehan Kwon et al. A 1ynm 1.25v 8gb 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep learning application. *IEEE*, 2023.

- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [10] Jarvis Labs. Disaggregated Prefill-Decode: The Architecture Behind Meta's LLM Serving — jarvislabs.ai. <https://jarvislabs.ai/blog/llm-optimization-disaggregated-prefill-decode>. [Accessed 27-04-2026].
- [11] Wantong Li, Madison Manley, James Read, Ankit Kaul, Muhannad S. Bakir, and Shimeng Yu. H3datten: Heterogeneous 3-d integrated hybrid analog and digital compute-in-memory accelerator for vision transformer self-attention. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 31(10):1592–1602, 2023.
- [12] Micron. Calculating Memory Power for DDR4 SDRAM. [Accessed 17-04-2026].
- [13] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, Rachata Ausavarungnirun, Mohammad Sadrosadati, and Geraldo F. Oliveira. A modern primer on processing in memory, 2025.
- [14] Rajeev Kumar Pandey and Sanjeev Kumar Pandey. Analyzing the performance of 7nm finfet based logic circuit for the signal processing in neural network. In *2020 IEEE Recent Advances in Intelligent Computational Systems (RAICS)*, pages 136–140, 2020.
- [15] Youngsuk Park, Kailash Budhathoki, Liangfu Chen, Jonas Kübler, Jiaji Huang, Matthäus Kleindessner, Jun Huan, Volkan Cevher, Yida Wang, and George Karypis. Inference optimization of foundation models on ai accelerators, 2024.
- [16] Janak Sharda, Madison Manley, Jungyoun Kwak, Chinsung Park, Muhannad Bakir, and Shimeng Yu. 3-d stacked hbm and compute accelerators for llm: Optimizing thermal management and power delivery efficiency. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, 11:116–122, 2025.
- [17] Cristina Silvano, Daniele Ielmini, Fabrizio Ferrandi, Leandro Fiorin, Serena Curzel, Luca Benini, Francesco Conti, Angelo Garofalo, Cristian Zambelli, Enrico Calore, Sebastiano Fabio Schifano, Maurizio Palesi, Giuseppe Ascia, Davide Patti, Nicola Petra, Davide De Caro, Luciano Lavagno, Teodoro Urso, Valeria Cardellini, Gian Carlo Cardarilli, Robert Birke, and Stefania Perri. A survey on deep learning hardware accelerators for heterogeneous hpc platforms, 2025.
- [18] Shrihari Sridharan, Jacob R. Stevens, Kaushik Roy, and Anand Raghunathan. X-former: In-memory acceleration of transformers. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 31(8):1223–1233, 2023.
- [19] Aaron Stillmaker. Scaling equations for the accurate prediction of cmos device performance from 180 nm to 7 nm. <http://vcl.ece.ucdavis.edu/pubs/2017.02.VLSIintegration.TechScale/VLSI-Scaling-Stillmaker.pdf>, 2017. [Accessed 20-04-2026].
- [20] Chirag Sudarshan. Processing-in-Memory DRAM Architectures for Neural Network Applications. [Accessed 17-04-2026].
- [21] Chirag Sudarshan, Taha Soliman, Cecilia De la Parra, Christian Weis, Leonardo Ecco, Matthias Jung, Norbert Wehn, and Andre Guntoro. A novel dram-based process-in-memory architecture and its implementation for cnns. In *2021 26th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 35–42, 2021.
- [22] Minchul Sung, Se-Aug Jang, Hyunjin Lee, Yun-Hyuck Ji, Jae-Il Kang, Tae-O Jung, Tae-Hang Ahn, Yun-Ik Son, Hyung-Chul Kim, Sun-Woo Lee, Seung-Mi Lee, Jung-Hak Lee, Seung-Beom Baek, Eun-Hyup Doh, Heung-Jae Cho, Tae-Young Jang, Il-Sik Jang, Jae-Hwan Han, Kyung-Bo Ko, Yu-Jun Lee, Su-Bum Shin, Jae-Seon Yu, Sung-Hyuk Cho, Ji-Hye Han, Dong-Kyun Kang, Jinsung Kim, Jae-Sang Lee, Keun-Do Ban, Seung-Jin Yeom, Hyun-Wook Nam, Dong-Kyu Lee, Mun-Mo Jeong, Byungil Kwak, Jeongsoo Park, Kisik Choi, Sung-Kye Park, Noh-Jung Kwak, and Sung-Joo Hong. Gate-first high-k/metal gate DRAM technology for low power and high performance products. In *2015 IEEE International Electron Devices Meeting (IEDM)*. IEEE, December 2015.
- [23] Fengbin Tu, Zihan Wu, Yiqi Wang, Ling Liang, Liu Liu, Yufei Ding, Leibo Liu, Shaojun Wei, Yuan Xie, and Shouyi Yin. Trancim: Full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. *IEEE Journal of Solid-State Circuits*, 58(6):1798–1809, 2023.
- [24] uplatzblog. Breaking the Memory Wall: An Architectural Analysis of Processing-in-Memory for Data-Intensive Computing — Uplatz Blog — uplatz.com. <https://uplatz.com/blog/breaking-the-memory-wall-an-architectural-analysis-of-processing-in-memory-for-data-intensive-computing/>. [Accessed 27-04-2026].
- [25] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. Efficient large language models: A survey. *Transactions on Machine Learning Research*, 2024. Survey Certification.
- [26] Wikipedia. Tensor Processing Unit - Wikipedia — en.wikipedia.org. https://en.wikipedia.org/wiki/Tensor_Processing_Unit. [Accessed 20-04-2026].
- [27] SK hynix Yongkee Kwon. Cost effective llm inference solution using sk hynix's aim (accelerator-in-memory). [Accessed 19-04-2026].
- [28] Shimeng Yu. Tutorial to NeuroSim: A Versatile Benchmark Framework for AI Hardware. <https://www.aspdac.com/aspdac2024/archive/pdf/T1-1.pdf>. [Accessed 17-04-2026].
- [29] Hengrui Zhang, Pratyush Patel, August Ning, and David Wentzlaff. Spad: Specialized prefill and decode hardware for disaggregated llm inference, 2025.
- [30] Youpeng Zhao, Di Wu, and Jun Wang. Alisa: Accelerating large language model inference via sparsity-aware kv caching, 2024.